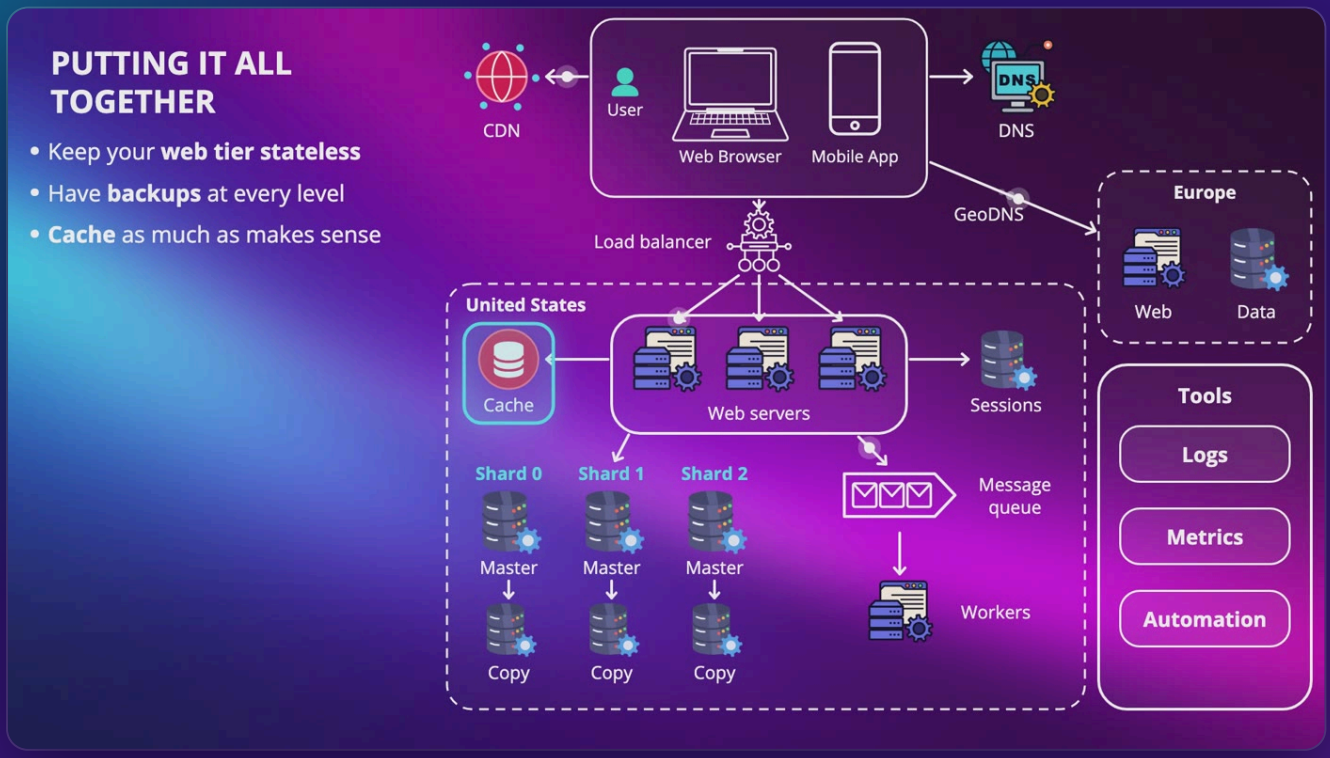
01

From One User to Millions

One bookstore, one problem at a time: every piece of a large system, and why it's there.



By Nata Vacheishvili · udemy.com/course/system-design-zero-to-hero

CHAPTER 1

What's inside

01 How a Request Reaches a Server: DNS, HTTP and APIs

02 Choosing a Database: Relational vs NoSQL

03 Vertical vs Horizontal Scaling

04 Load Balancers: Keeping Your Servers Online

05 Database Replication: Surviving a Database Failure

06 Caching: Making Your System Faster

07 CDNs: Delivering Files Closer to Your Users

08 Stateless Servers and Data Centers

09 Message Queues, Logging and Metrics

10 Sharding and Putting It All Together

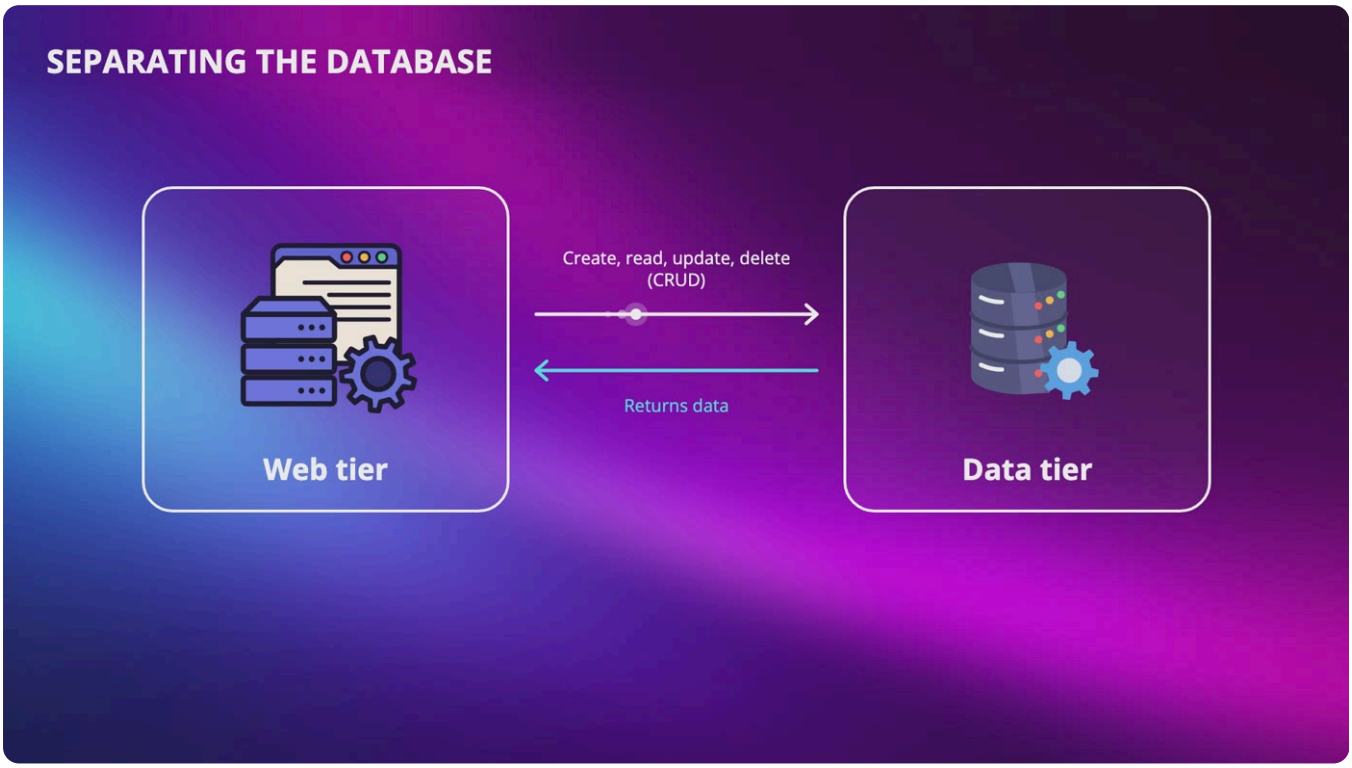
How to use these notes

- Watch the lecture first, then use its page to review.
- The big idea is the one thing to take away.
- Terms you can't remember are in the New Terms box, and the index at the end shows which lecture covers each one.

01 How a Request Reaches a Server: DNS, HTTP and APIs

THE BIG IDEA

Nobody builds for millions on day one. Start with one server and one user, then grow one problem at a time.



We split one server into a web tier and a data tier, so each can grow on its own.

Following one request

- Your phone asks **DNS** for the address of the bookstore, and gets back an **IP address**.
- Your phone sends an **HTTP request** directly to the web server.
- The server answers with an **HTML** page for a browser, or **JSON** data for a mobile app, which already has its own screens.

Why split the database out?

- One server was doing two very different jobs: answering every visitor, and storing and searching all the data.
- Once they're separate, they grow independently. More visitors but no new books? Give more power to the web tier only.

IN THE REAL WORLD

In October 2021, Facebook, Instagram and WhatsApp went down. A big part of the reason was that Facebook's DNS servers became unreachable. The machines were there, but nobody could find them.

New terms

Server
A computer, or a program on one, that provides services, data or resources to other computers.

Client
The computer or app asking the server for something.

IP address
A set of numbers that tells other machines where to find a machine, like a home address.

DNS
Domain Name System. Translates names like google.com into IP addresses, like the contacts on your phone.

HTTP request
A message your computer sends to a web server asking for something.

HTML
The file that describes what a page looks like.

JSON
JavaScript Object Notation, a simple text format for sending data.

API
A way for one program to request something from another.

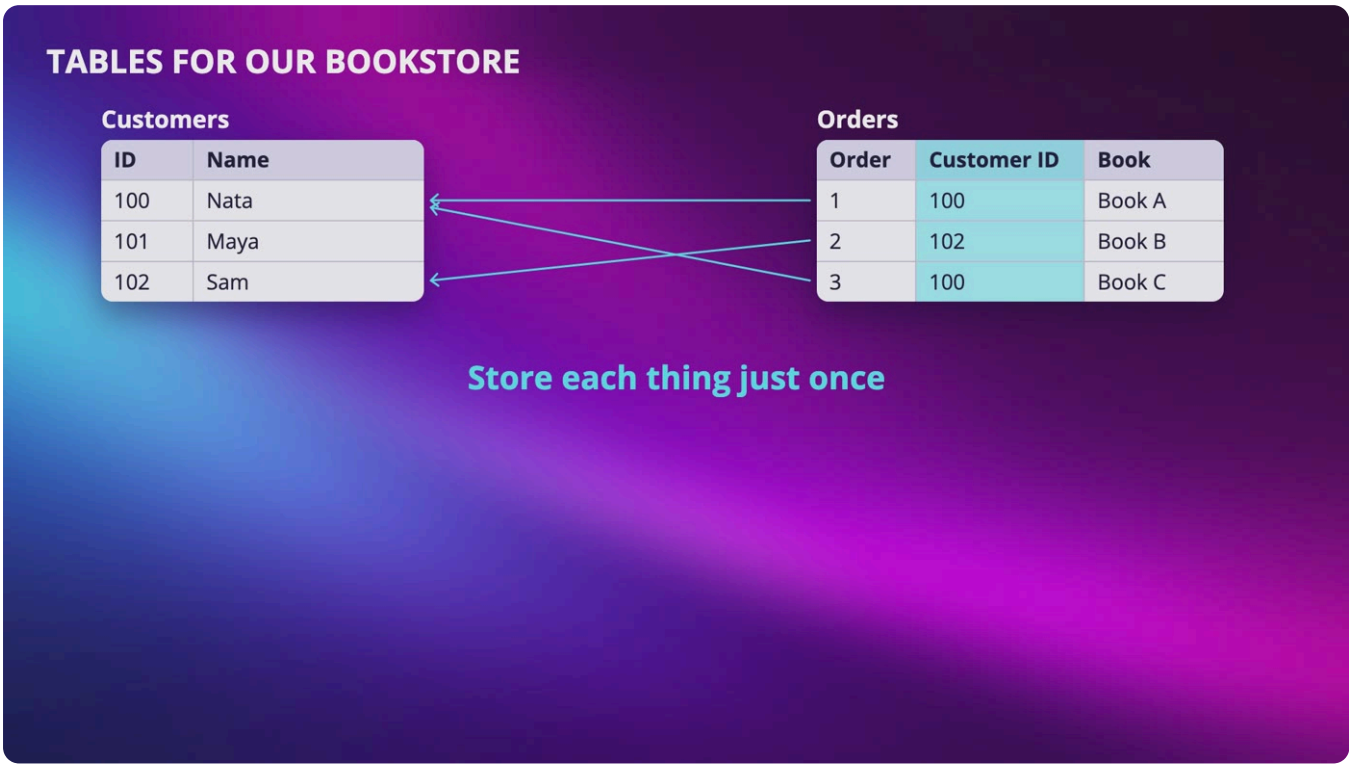
Web tier / data tier
The layer handling web and mobile traffic, and the layer holding the database.

CRUD
Create, read, update and delete.

02 Choosing a Database: Relational vs NoSQL

THE BIG IDEA

Don't memorize a list. Look at your data and how it will be used, then choose. Most developers should start relational.



Customers and orders have relationships, so they fit tables. Each thing is stored just once.

Compare them

- **Relational** stores tables and rows, and joins let you ask questions you didn't plan for. Think customers and orders.
- **Non-relational** generally gives up joins to get something else. Think a shopping cart, which you grab whole by user ID, and which must be fast and always work.

Four types of NoSQL

- **Key-value:** look something up by its key, like a dictionary.
- **Document:** each item is a document, usually something like JSON.
- **Column:** groups related columns to handle huge amounts of data efficiently.
- **Graph:** keeps the connections between things, like who's friends with who.

Good reasons to go non-relational

- You need super low latency.
- Your data is unstructured, or has no relations.
- You only need to serialize and deserialize your data.
- You need to store a massive amount of data.

It isn't all or nothing

- Our bookstore can keep customers and orders in a relational database, and the cart in a key-value store.

IN YOUR INTERVIEW

Don't just say "I'd use NoSQL because it scales." Look at the data, ask how it's going to be used, and explain your choice out loud.

IN THE REAL WORLD

Amazon wrote that customers should be able to add to their cart even if disks are failing or a data center is destroyed by a tornado. That's a big part of why it built its own key-value store, Dynamo.

New terms

Relational database (RDBMS)
Stores data in tables and rows, like a spreadsheet. Also called a SQL database.

Join
Combines rows from two or more tables based on a related column.

Latency
The delay between sending a request and receiving the response.

Serialization
Converting something, like an object in your code, into a format that can be stored or sent, like JSON. Deserialization is the opposite.

SQL
The language you use to talk to a relational database.

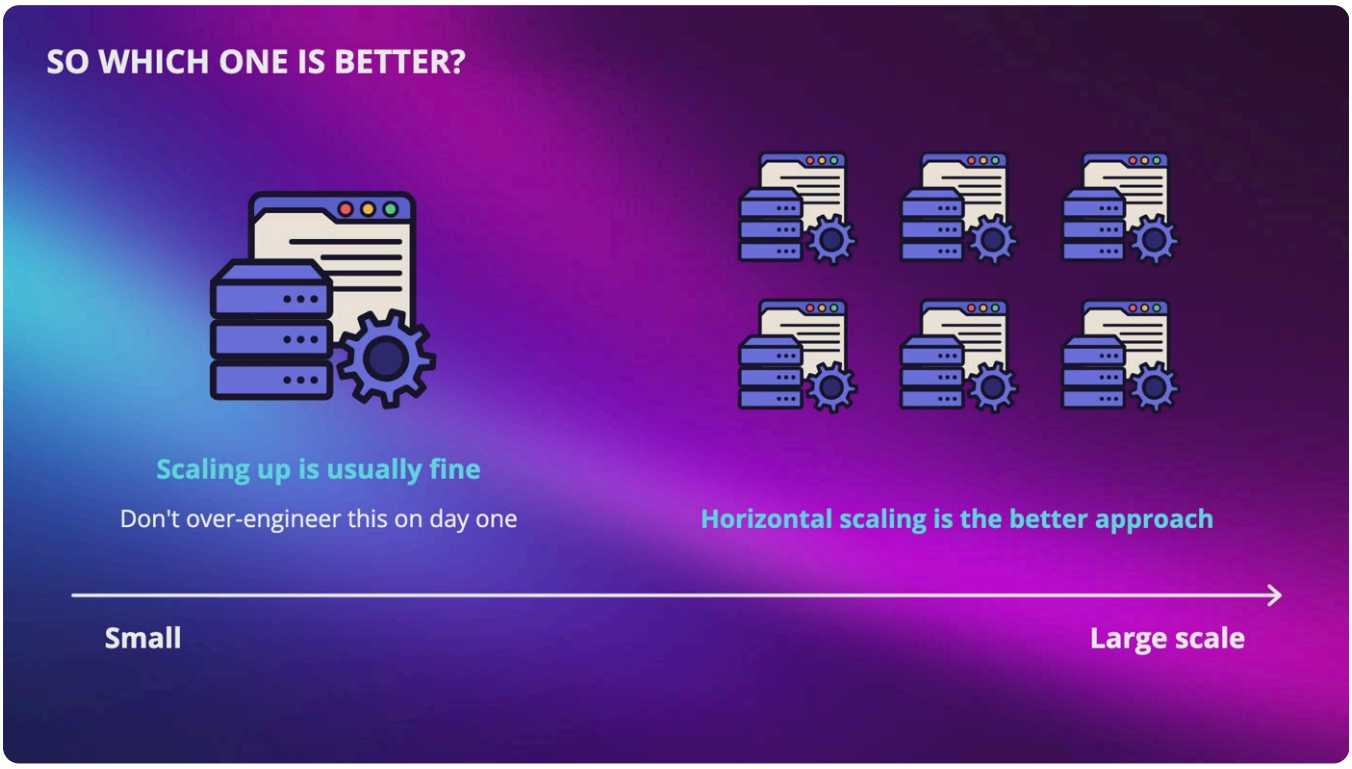
NoSQL
A non-relational database. Examples: CouchDB, Neo4j, Cassandra, DynamoDB.

Unstructured data
Data that doesn't fit neatly into tables.

03 Vertical vs Horizontal Scaling

THE BIG IDEA

Scaling up gives you one stronger machine. Scaling out gives you more machines, and only that removes the single point of failure.



When you're small, scaling up is usually fine. At large scale, horizontal scaling is the better approach.

	Scale up (vertical)	Scale out (horizontal)
What you do	Add CPU or RAM to your server	Add more servers to the pool
Like	One much faster chef	Hiring more chefs
Best part	Simple, with no code changes	Keeps growing, and survives a crash
Worst part	Hard limit, and if it dies you're offline	Several machines to manage
Use it	When you're small	For large scale applications

IN YOUR INTERVIEW

Don't jump to the most complicated answer. Say what you'd start with, say when it would stop working, and then say what you'd do next.

New terms

CPU
Central processing unit. The brain of a computer, which runs instructions and calculations.

RAM
Random access memory. The computer's short-term memory.

Single point of failure
One part of a system that takes the whole system down when it fails.

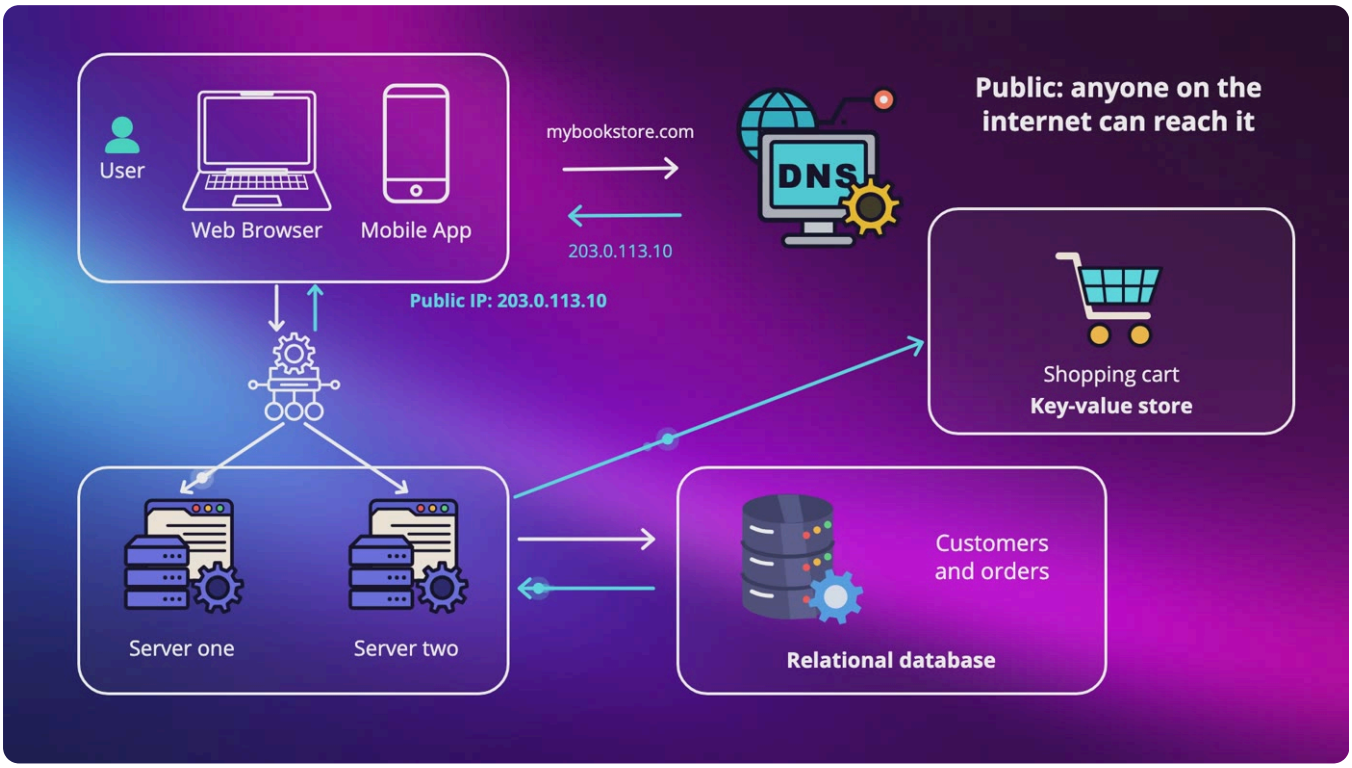
Failover
If one machine fails, another takes over. Swapping in the spare tire.

Redundancy
Having multiple copies of a resource. Having the spare tire.

04 Load Balancers: Keeping Your Servers Online

THE BIG IDEA

A load balancer sits in front of a pool of servers and spreads requests across them, so one crash doesn't take you down.



Users connect to the load balancer's public IP. Only the load balancer talks to the web servers.

What it gives us

- If server one goes down, the load balancer sends everyone to server two.
- If traffic grows, add servers. The address users connect to never changes.
- It knows a server is down through **health checks**. If a server stops answering, it stops getting traffic.

Watch out

- **Failover isn't instant.** For a short moment, some users can still see errors.
- **Keep spare room.** Two servers at 80%, and one dies? The other gets 160% of what it can handle and goes down too.
- **One load balancer is itself a single point of failure.** Usually we run two, and cloud providers can handle this for you.

IN YOUR INTERVIEW

Don't just draw the box and move on. Interviewers can ask what happens if it dies, and how it knows a server is down. And don't say failover is instant.

New terms

Load balancer
Takes every incoming request and spreads the requests across servers. Like the host at a busy restaurant.

Pool
The group of servers behind a load balancer.

Public IP
An address anyone on the internet can reach.

Private IP
An address that only works between machines on the same private network.

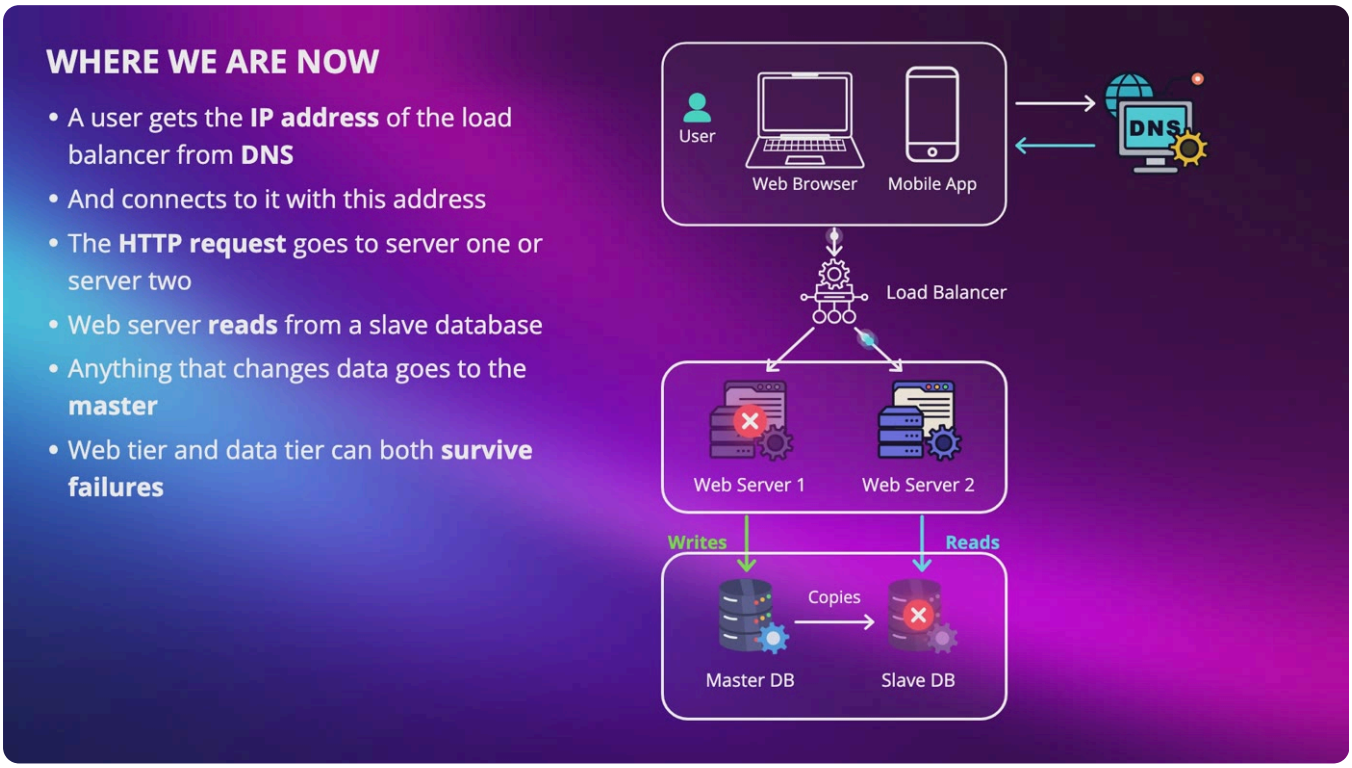
Subnet
A range of addresses that defines which network machines belong to.

Health check
A small request sent to each server on a schedule, to see if it's still answering.

05 Database Replication: Surviving a Database Failure

THE BIG IDEA

Keep copies of your database. All writes go to one place, and reads are spread across the copies.



Where we are now: writes go to the master, reads come from the slave, and both tiers can survive failures.

Why only one place for writes?

- The master is the single source of truth. If two customers bought the last copy on different databases, both could be told yes.
- Most apps have many more reads than writes, so there are usually more slaves than masters.

What it gives us

- **Better performance:** more queries can run in parallel.
- **Reliability:** data survives if a server is destroyed.
- **High availability:** the app stays up if one database is offline.

When a database goes offline

- **One slave down:** reads go temporarily to the master until a new slave replaces it.
- **One of many slaves down:** reads go to the healthy slaves.
- **Master down:** a slave gets promoted to be the new master. Copies take time, so it might be missing some data, which makes this very complicated in production.

IN THE REAL WORLD

In October 2018, GitHub lost the connection between two locations for 43 seconds. Its primary databases moved automatically, and neither side had the full picture. The result was 24 hours and 11 minutes of degraded service, with no user data lost in the end.

New terms

Database replication
Keeping copies of your database on more than one server.

Master / slave
The original, which takes writes, and the copies, which serve reads. Also called primary and replica.

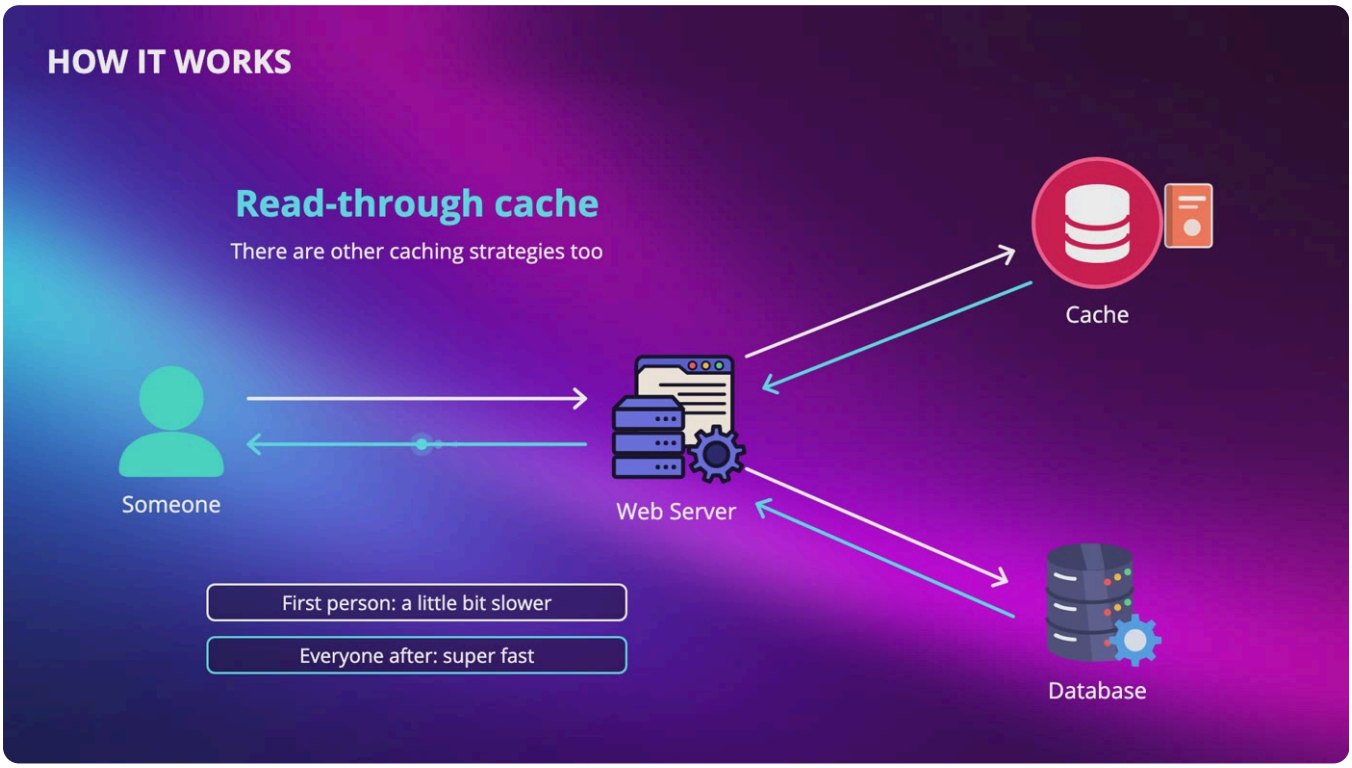
Write operations
Anything that changes data, like inserts, updates and deletes.

Read operations
Anything that only looks at data.

06 Caching: Making Your System Faster

THE BIG IDEA

Keep frequently used answers in memory so that you don't ask the database the same question again and again.



The web server checks the cache first, and only goes to the database if the answer isn't there.

Read-through cache

- The server checks the cache first. If the data's there, it sends it right back.
- If not, it goes to the database, saves the answer in the cache, then sends it back.
- The first person is a little slower, and everyone after them gets the fast answer.

Five things to think about

- **When to use it:** data that's read often and changed rarely. Never keep the only copy there, because memory is volatile.
- **Expiration:** not too short, or you lose the benefit. Not too long, or data goes stale.
- **Consistency:** if the price changes in the database but not the cache, a customer can see one price and be charged another.
- **Failures:** one cache server is a single point of failure, so use several.
- **Eviction:** when the cache is full, something has to go.

IN THE REAL WORLD

Facebook published a paper on its use of Memcached, a caching system, saying it handled billions of requests per second.

New terms

Cache
Temporary storage for frequently used data, or responses that were expensive to get.

Persistent data store
One that keeps data even when the power goes off.

Eviction
Removing an existing item to make room for a new one.

Multi-region
Deployed in two or more geographic locations around the world.

Volatile
Lost when a server restarts, as data in memory is.

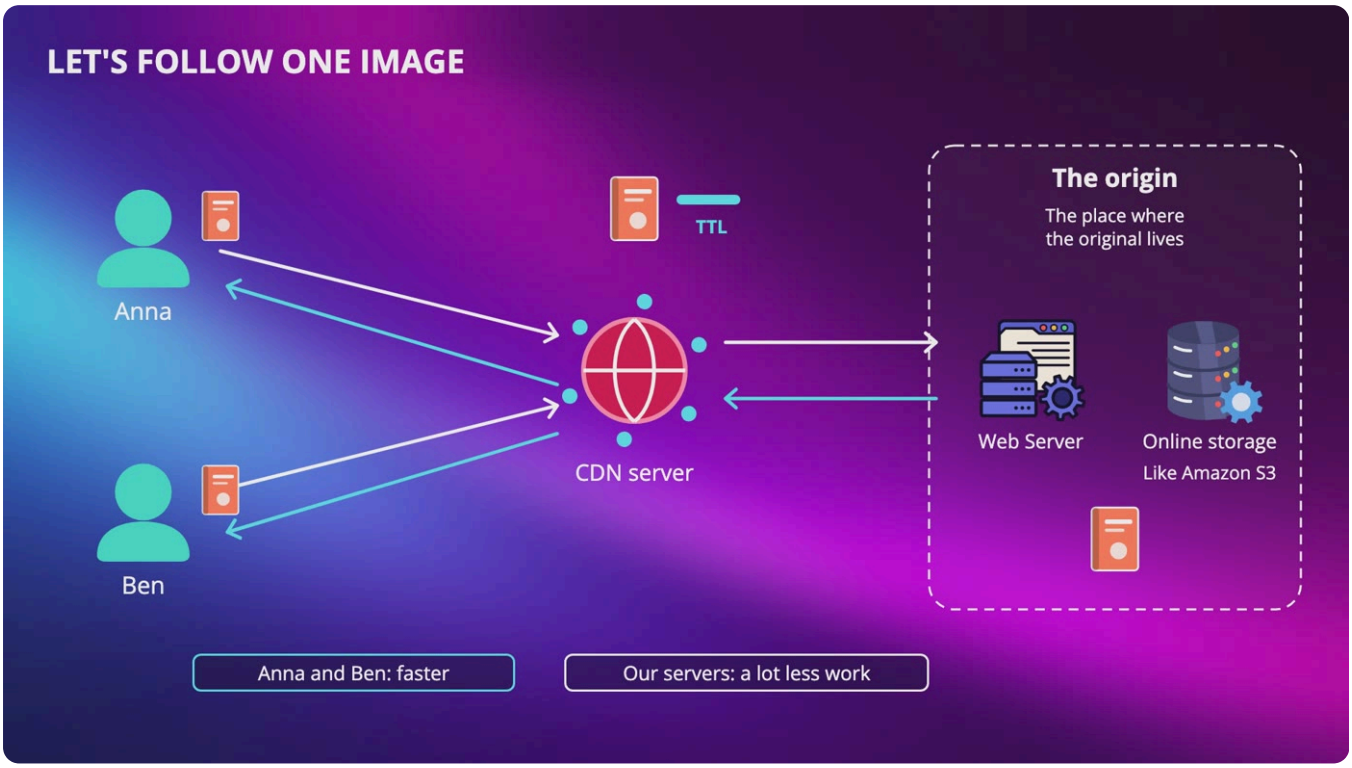
Stale data
Data that's out of date.

LRU
Least recently used. Removes whatever hasn't been used for the longest time.

07 CDNs: Delivering Files Closer to Your Users

THE BIG IDEA

The closer the server is to the user, the faster the file loads. A CDN keeps copies of static files all around the world.



Anna's request gets the file from the origin. Ben, nearby, then gets it straight from the CDN.

How it works

- The image address points to the CDN, and the request goes to the nearest CDN server.
- The first time, the CDN gets the file from the origin, keeps a copy and sends it.
- The next nearby user gets it straight from the CDN, as long as the TTL hasn't expired.

What to consider

- **Cost:** you pay for data transfers, so don't cache rarely used files.
- **Expiry:** not too long, and not too short.
- **Fallback:** if the CDN has an outage, clients should request from the origin.
- **Changing a file: invalidate** it through your provider's API, or use **versioning**, like cover.jpeg?v=2.

IN THE REAL WORLD

Netflix's Open Connect program partners with over a thousand internet service providers to put content closer to viewers.

New terms

CDN
Content delivery network. Servers spread across locations that deliver static content.

CSS
The code that controls how a website looks.

TTL
Time to live. How long a file should stay cached.

Versioning
Adding a version number to a file's address, so the CDN treats it as new.

Static content
Files that are the same for everyone, like images, videos, CSS and JavaScript.

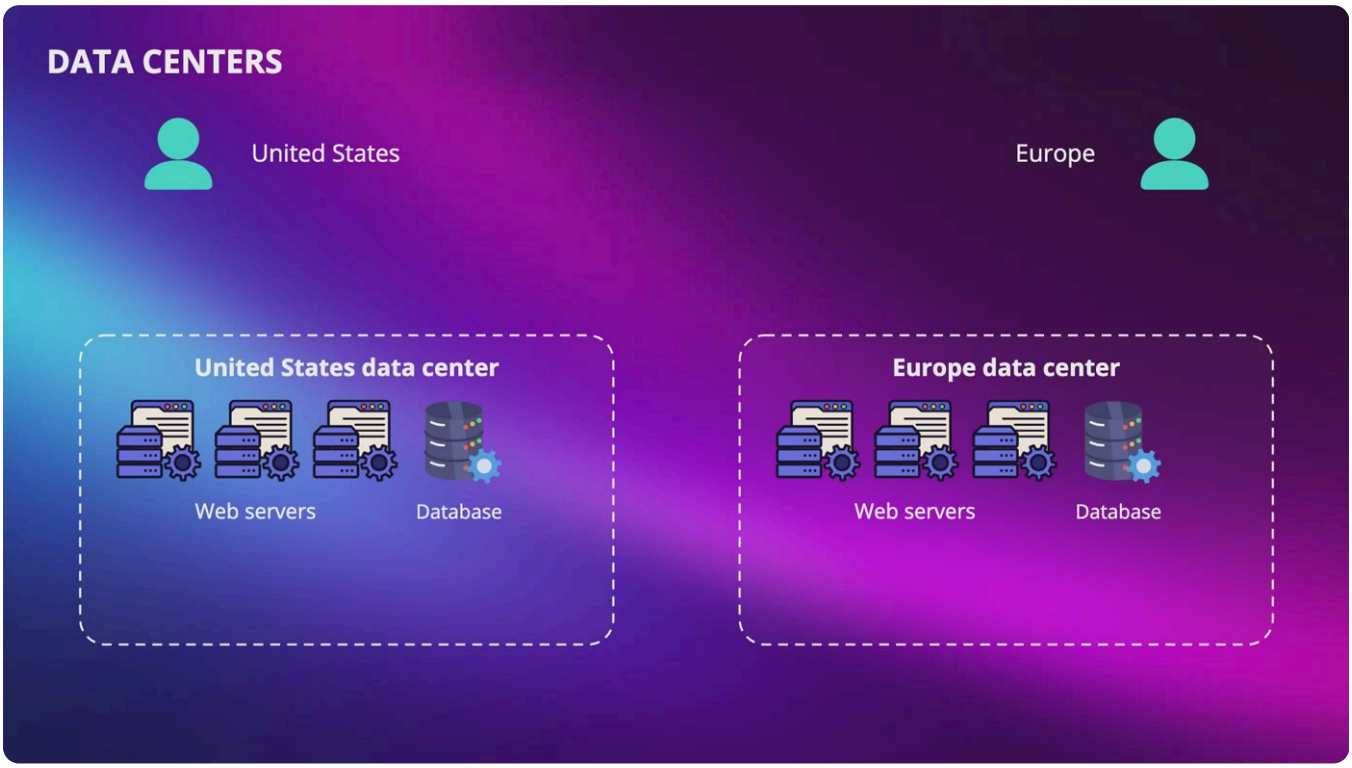
Origin
The place where the original file lives, like your web server or Amazon S3.

Invalidate
Tell the CDN to remove a file.

08 Stateless Servers and Data Centers

THE BIG IDEA

Keep your web tier stateless, so that servers can come and go. Add data centers when you actually need them.



Users are sent to the closest data center, and traffic can move if one has an outage.

Stateful vs stateless

- **Stateful:** your session is on server one. The load balancer sends you to server two, and it looks like you got logged out.
- **Sticky sessions** keep you on one server, but adding or removing servers gets harder, and a crash loses sessions.
- **Stateless:** move sessions to a shared data store. Any server can help you, and auto scaling becomes much easier.

Data center challenges

- **Traffic redirection:** sending traffic to the right place, where GeoDNS helps.
- **Data synchronization:** if Europe goes down and its orders exist only there, users see no orders. Replicating helps, but copies take time.
- **Testing and deployment:** automatic deployment tools keep every location consistent.

IN YOUR INTERVIEW

Don't draw multiple data centers just because it looks impressive. Ask how many users there are and where they are first.

IN THE REAL WORLD

On October 20, 2025, Amazon Web Services had a big outage in its Northern Virginia region. If your whole system lives in one place, its bad day is your bad day.

New terms

State / session data
Information a server remembers about a user, like that you're logged in.

Stateful server
Remembers client data from one request to the next.

Stateless server
Keeps no state information.

Sticky sessions
The load balancer sends every request from one client to the same server.

Redis
An in-memory database that's very fast.

Auto scaling
Adding or removing web servers automatically based on traffic.

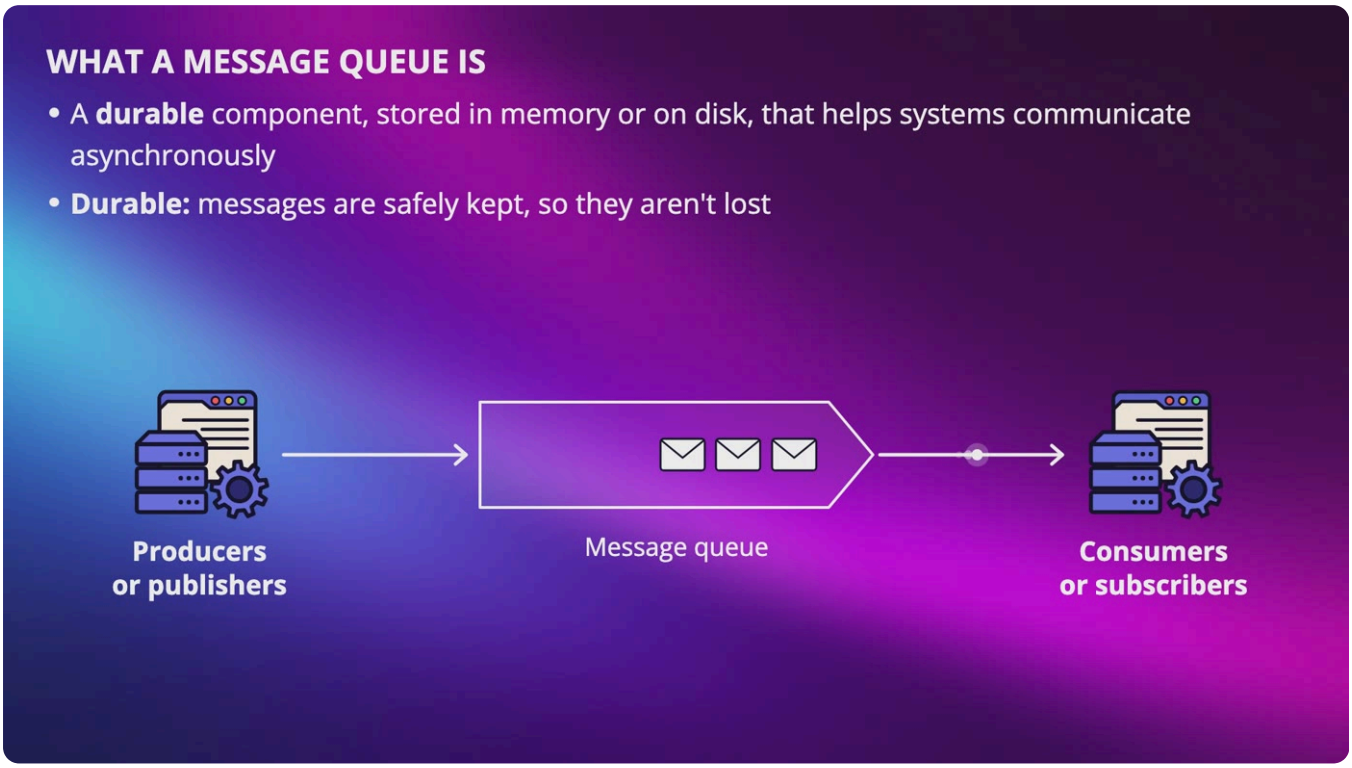
Data center
A building full of servers.

GeoDNS
A DNS service that gives back a different IP address based on the user's location.

09 Message Queues, Logging and Metrics

THE BIG IDEA

Slow work shouldn't hold anyone up. A queue lets the parts of a system work at different times, and logs and metrics show you what's happening.



Producers drop messages in the queue, and consumers pick them up whenever they're available.

Why a queue helps

- A purchase saves the order and puts "send a confirmation email" in the queue, then immediately thanks the customer.
- If the email service is down for ten minutes, messages just wait, and nothing is lost.
- Producers and consumers scale independently. When the queue gets large, add workers. If it's mostly empty, use fewer and save money.

Seeing inside your system

- **Logging** helps you detect errors. With many servers, bring logs into one central place.
- **Metrics** show health and business insights. Metrics tell you something's wrong, and logs help you find out what.
- **Automation** means fewer mistakes, by automating building, testing and deploying.

New terms

Synchronous
The sender waits until the receiver is done. Like calling a friend.

Asynchronous
Sender and receiver don't have to work at the same time. Like texting a friend.

Message queue
A durable component that helps systems communicate asynchronously. Like a mailbox.

Durable
Messages are safely kept, so they aren't lost.

Producer / consumer
Producers (publishers) create messages, and consumers (subscribers) read them and do the work.

Decoupling
The sender doesn't care how fast the receiver is, or whether it's online.

Log
A record of what happened.

Metrics
Measurements over time, like CPU, or daily active users.

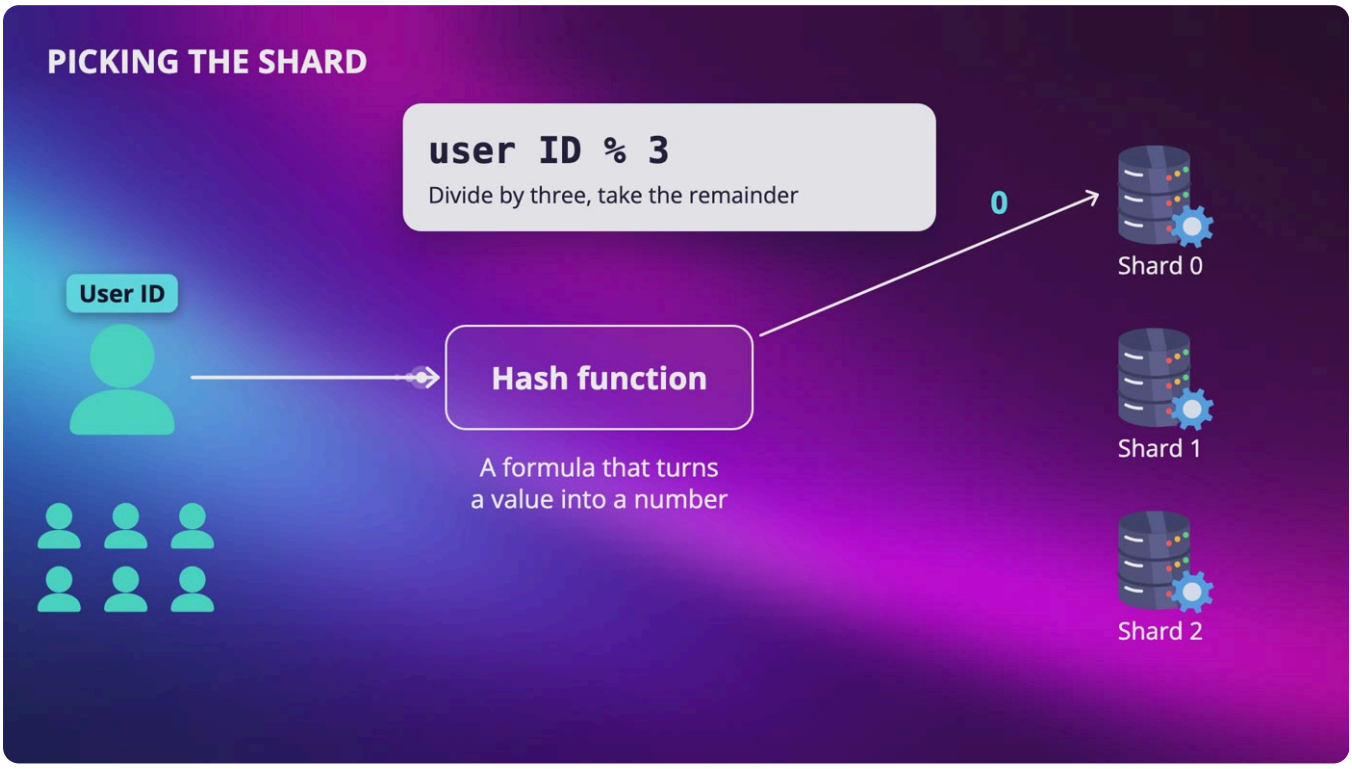
Retention
How many of your users keep coming back.

Continuous integration
Each code check-in is automatically verified, so teams detect problems early.

10 Sharding and Putting It All Together

THE BIG IDEA

When data outgrows one database, split it into shards. Your sharding key is the most important decision you'll make here.



A hash function sends each user to the right shard: user ID modulo 3. User seven goes to shard one.

Choosing a sharding key

- It lets you send each query straight to the right database, instead of asking all of them.
- Pick one that spreads data evenly. Sharding by country could leave one shard with almost everyone.

Why sharding is hard

- **Resharding:** moving from three shards to four means most users suddenly belong elsewhere. Consistent hashing helps, later in the course.
- **The celebrity problem:** Adele and other celebrities on one shard overwhelm it, while others stay quiet.
- **Joins:** hard across servers. A common workaround is denormalization.

IN YOUR INTERVIEW

Know the complexities of sharding for your interviews: resharding, the celebrity problem and joins.

IN THE REAL WORLD

Notion wrote about splitting its Postgres database, settling on 480 logical shards across 32 physical databases.

New terms

Sharding
Horizontal scaling for databases. Splitting a large database into smaller parts called shards.

Hash function
A formula that turns a value into a number.

Sharding key
One or more columns that determine how data is distributed. Also called a partition key.

Hotspot key
Another name for the celebrity problem.

Schema
The tables and columns. Each shard has the same schema, with different data.

Modulo
Dividing and taking the remainder.

Shard exhaustion
When some shards fill up much faster than others.

Denormalization
Intentionally adding duplication, so queries don't have to join many tables.

CHAPTER RECAP

How our bookstore grew

You grow, something breaks, and you fix it.

Every piece solved a real problem

One server	You're the only user
Separate database	One machine had two jobs
Load balancer	One crash took us down
Replication	One database could lose everything
Cache	Same slow question every time
CDN	Files traveled across the world
Stateless servers	Servers couldn't come and go
Data centers	Customers around the world
Message queue	Slow work held people up
Logs and metrics	We couldn't see inside
Shards	Data got too big

What to remember

- ✓ Keep your web tier stateless
- ✓ Have backups at every level
- ✓ Cache as much as makes sense
- ✓ Serve your static files from a CDN
- ✓ Think about multiple data centers when you actually need them
- ✓ Shard your data tier when it gets too big
- ✓ Split things into smaller services as you grow
- ✓ Always monitor your system

FIND A TERM

Index of new terms

The number is the lecture in this chapter that explains it.

API	1	Invalidate	7	Relational database (RDBMS)	2
Asynchronous	9	IP address	1	Retention	9
Auto scaling	8	Join	2	Schema	10
Cache	6	JSON	1	Serialization	2
CDN	7	Latency	2	Server	1
Client	1	Load balancer	4	Shard exhaustion	10
Continuous integration	9	Log	9	Sharding	10
CPU	3	LRU	6	Sharding key	10
CRUD	1	Master / slave	5	Single point of failure	3
CSS	7	Message queue	9	SQL	2
Data center	8	Metrics	9	Stale data	6
Database replication	5	Modulo	10	State / session data	8
Decoupling	9	Multi-region	6	Stateful server	8
Denormalization	10	NoSQL	2	Stateless server	8
DNS	1	Origin	7	Static content	7
Durable	9	Persistent data store	6	Sticky sessions	8
Eviction	6	Pool	4	Subnet	4
Failover	3	Private IP	4	Synchronous	9
GeoDNS	8	Producer / consumer	9	TTL	7
Hash function	10	Public IP	4	Unstructured data	2
Health check	4	RAM	3	Versioning	7
Hotspot key	10	Read operations	5	Volatile	6
HTML	1	Redis	8	Web tier / data tier	1
HTTP request	1	Redundancy	3	Write operations	5